Especificación de mensajes MQTT

Sistema de reserva de turnos | GrupoIIS2026 | Parte 6

Versión 1.1 · 6 de noviembre de 2026. Contratos obtenidos de las clases Java y DTO del proyecto proporcionado. Los ejemplos son ilustrativos: deben utilizarse identificadores existentes y horarios adecuados al ejecutar pruebas.

1. Conexión y convenciones

Broker: Eclipse Mosquitto. Desde los contenedores se utiliza tcp://mosquitto:1883, configurable mediante BROKER_URL. Desde la computadora se utiliza localhost:1883. La configuración de demostración permite conexiones anónimas y no configura TLS.

Cada payload es un objeto JSON. Se deben enviar textos en UTF-8. EventosTurnosMQTT lo especifica expresamente; las otras clases usan la codificación predeterminada de Java. Los identificadores son enteros de 64 bits (Long); teléfonos y correos son cadenas. Las fechas y horas se interpretan sin desplazamiento horario, con la configuración America/Montevideo de los contenedores.

2. Catálogo de tópicos

Tópico	Publicador → consumidor
turnos/reserva	Generador o cliente MQTT → app-listener
turnos/solicitudes	api-rest → procesador-solicitudes
turnos/cancelar	api-rest o cliente MQTT → app-listener
turnos/atendidos	procesador-solicitudes → facturador
turnos/disponibilidad	Cliente MQTT → app-listener
turnos/disponibilidad/respuesta	app-listener → cliente suscrito
turnos/agenda	Cliente MQTT → app-listener
turnos/agenda/respuesta	app-listener → cliente suscrito

3. Identificación de operaciones

El tópico identifica la operación. Estos contratos no usan el envoltorio antiguo con status, fechaHora y turno. No se deben intercambiar personalId, idPersonal y reservaId: cada mensaje utiliza exactamente los nombres descritos en las páginas siguientes.

Reserva y aviso de solicitud

4. turnos/reserva

Finalidad: solicitar una reserva enviando sus datos completos. Publicador: GeneradorReservasMQTT o cliente MQTT. Consumidor: SubscriberReservaMQTT (app-listener). El generador publica cada 10 segundos.

Campo / tipo	Descripción y condición
emailSolicitante · string	Correo del cliente. Debe enviarse; esta vía no contiene una validación explícita de presencia o formato del correo.
telefonoSolicitante · string	Teléfono del cliente, conservando ceros iniciales. Debe enviarse; esta vía no valida explícitamente su presencia.
personalId · integer	Identificador de un empleado existente, activo y con establecimiento asignado. Obligatorio.
fechaHoraTurno · string	Fecha y hora local ISO: yyyy-MM-ddTHH:mm o yyyy-MM-ddTHH:mm:ss. Obligatorio.

```
{
  "emailSolicitante": "cliente@example.com",
  "telefonoSolicitante": "099123456",
  "personalId": 2,
  "fechaHoraTurno": "2026-12-10T11:00:00"
}
```

El establecimiento se obtiene del personal; este mensaje no requiere establecimientoId. El ID de reserva y la fecha de registro se generan al persistir. El listener comprueba personal, duración de 30 minutos, horario de atención y solapamiento. El procesamiento posterior determina el estado de la solicitud. No se publica un acuse de negocio en otro tópico: el resultado se verifica en registros, API o base de datos.

5. turnos/solicitudes

Finalidad: avisar que existe una reserva para evaluar. La API publica el mensaje después de saveAndFlush; el procesador lee la reserva desde MySQL. No es otro mensaje de alta con los datos completos del cliente.

Campo / tipo	Descripción y condición
reservaId · integer	ID positivo de una reserva existente en estado SOLICITADO. Obligatorio.

```
{
  "reservaId": 11
}
```

El procesador ejecuta su ciclo cada 60 segundos, según VALIDATION_INTERVAL_SECONDS. Además de los avisos recibidos, busca reservas SOLICITADO en MySQL. Asigna AGENDADO o un estado de rechazo según las validaciones. Un aviso por sí solo no confirma el turno.

Cancelación y aviso de atención

6. turnos/cancelar

Finalidad: solicitar la cancelación de una reserva. Publicadores: API REST al recibir DELETE /api/reservas/{id}, o cliente MQTT. Consumidor: app-listener.

Campo / tipo	Descripción y condición
reservaId · integer	Identificador positivo de la reserva que se desea cancelar. Obligatorio.

```
{
  "reservaId": 11
}
```

Si la reserva está SOLICITADO o AGENDADO, se conserva el registro y pasa a CANCELADO. Repetir la cancelación de una reserva ya CANCELADO no vuelve a modificarla. Un ID inexistente o un estado distinto de los permitidos genera un rechazo en los registros del listener.

No existe un tópico de respuesta de cancelación en esta implementación. El resultado se comprueba consultando GET /api/reservas/{id}, la base o los registros. La aceptación HTTP 202 de la API indica recepción para procesamiento, no cancelación ya completada.

7. turnos/atendidos

Finalidad: avisar que un turno fue marcado como atendido. Publicador: ProcesadorSolicitudes. Consumidor: FacturadorAutomatico.

Campo / tipo	Descripción y condición
reservaId · integer	ID positivo de una reserva existente en estado ATENDIDO. Obligatorio.

```
{
  "reservaId": 11
}
```

El procesador marca un turno AGENDADO como ATENDIDO cuando su fecha y hora de finalización no son posteriores al momento actual. Publica el aviso después de confirmar la transacción de base de datos.

El facturador procesa los avisos en su ciclo de 300 segundos, configurable mediante BILLING_INTERVAL_SECONDS. También consulta pendientes ATENDIDO en MySQL. Crea o recupera el cliente y la factura mensual, registra el ítem y cambia el estado a FACTURADO.

Este tópico no crea reservas ni debe usarse como un comando para forzar atención: el facturador vuelve a comprobar el estado almacenado. No hay un tópico de respuesta de facturación en el código revisado.

Consulta de disponibilidad

8. turnos/disponibilidad

Publicador: cliente MQTT. **Consumidor:** app-listener. Consulta si el personal puede atender un turno de 30 minutos a partir de la fecha y hora solicitadas.

Campo / tipo	Descripción y condición
personalId · integer	Identificador del empleado consultado. Obligatorio.
fechaHoraTurno · string	Fecha y hora local ISO, sin zona u offset. Obligatorio.

```
{
  "personalId": 2,
  "fechaHoraTurno": "2026-12-10T11:00:00"
}
```

9. turnos/disponibilidad/respuesta

Publicador: app-listener. **Consumidor:** cualquier cliente previamente suscrito al tópico de respuesta.

Campo / tipo	Descripción y condición
personalId · integer	Identificador recibido en la consulta.
fechaHoraTurno · string	Fecha y hora recibidas en la consulta.
disponible · boolean	true si la comprobación permite el horario; false en caso contrario o ante datos inválidos.
motivo · string	Texto explicativo del resultado.

```
{
  "personalId": 2,
  "fechaHoraTurno": "2026-12-10T11:00:00",
  "disponible": true,
  "motivo": "Horario disponible"
}
```

Si no está disponible, la estructura se conserva con disponible=false. Los motivos implementados incluyen datos faltantes, fecha/hora inválida, personal inexistente o inactivo, falta de establecimiento, duración distinta de 30 minutos, horario fuera de atención y turno solapado.

La disponibilidad es una consulta al estado actual; no bloquea el horario ni garantiza que una reserva posterior sea aceptada. Los campos con valor null pueden omitirse en las respuestas serializadas por Gson.

Consulta de agenda

10. turnos/agenda

Publicador: cliente MQTT. **Consumidor:** app-listener. Consulta las reservas de un empleado para una fecha.

Campo / tipo	Descripción y condición
personalId · integer	Identificador del empleado consultado. Obligatorio.
fecha · string	Fecha ISO yyyy-MM-dd. Obligatorio.

```
{
  "personalId": 2,
  "fecha": "2026-12-10"
}
```

11. turnos/agenda/respuesta

Publicador: app-listener. **Consumidor:** cliente suscrito al tópico de respuesta.

Campo / tipo	Descripción y condición
personalId / fecha	Valores de la consulta.
cantidad · integer	Número de elementos de turnos. Presente en respuesta exitosa.
turnos · array	Lista ordenada por fecha y hora; vacía si no hay resultados. Cada elemento contiene id (integer), hora (string), emailSolicitante y telefonoSolicitante (string).
error · string	Presente ante datos faltantes, fecha inválida o personal inexistente. En estos casos no se incluyen cantidad ni turnos.

```
{
  "personalId": 2,
  "fecha": "2026-12-10",
  "cantidad": 1,
  "turnos": [
    {
      "id": 11,
      "hora": "11:00",
      "emailSolicitante": "cliente@example.com",
      "telefonoSolicitante": "099123456"
    }
  ]
}
```

Ejemplo de error:

```
{
  "personalId": 999,
  "fecha": "2026-12-10",
  "error": "El personal no existe"
}
```

Entrega, errores y verificación

12. Propiedades del transporte

Los publicadores revisados utilizan QoS 1: pueden producirse reentregas. El acuse MQTT confirma entrega en el protocolo; no confirma reserva agendada, cancelación completada ni factura emitida. No se utilizan mensajes retained para estos contratos. El listener usa subscribe(topic) sin QoS explícito; EventosTurnosMQTT se suscribe con QoS 1.

La API conecta con sesión limpia. El generador y el listener conectan con opciones predeterminadas. EventosTurnosMQTT usa cleanSession=false, reconexión automática e IDs fijos para procesador y facturador; almacena los pendientes del proceso en memoria. Estas opciones no garantizan por sí solas recuperar todos los mensajes tras una caída. Los ciclos de consulta a MySQL complementan los avisos de solicitudes y atención.

13. Reintentos y correlación

Los avisos internos agrupan IDs pendientes en un conjunto; se comprueba el estado de la reserva antes de procesarla. Cancelar dos veces una reserva ya cancelada es tolerado. La restricción única de ITEMFACTURA.reserva_id impide registrar dos ítems para la misma reserva. El alta en turnos/reserva no incorpora un identificador de solicitud que garantice idempotencia: reenviar el mismo JSON no tiene una garantía general de no duplicación.

Las consultas de agenda y disponibilidad comparten tópicos de respuesta. No existe idSolicitud, responseTopic ni correlationId. Para asociar una respuesta se comparan personalId y fecha o fechaHoraTurno. Dos consultas idénticas concurrentes no pueden distinguirse con el contrato actual. Debe suscribirse el cliente antes de publicar la consulta.

14. Tratamiento de errores y alcance

Los avisos internos con JSON inválido o reservaId ausente/no positivo se descartan y registran. La cancelación también controla JSON e ID inválidos. Agenda y disponibilidad publican errores de validación para los casos descritos, pero no hay un sobre de error uniforme para cualquier JSON malformado. Los errores de alta se informan por registros; no hay tópico de mensajes fallidos.

Las consultas MQTT de agenda y disponibilidad filtran los estados AGENDADO, ATENDIDO y FACTURADO. Excluyen solicitudes pendientes, rechazadas y canceladas. La agenda no devuelve el estado de cada turno. Se verificó el caso de la reserva 11, CANCELADO, del personal 6 para el 4 de noviembre de 2026 a las 11:00: disponibilidad respondió true y la agenda devolvió turnos=[] y cantidad=0.

15. Observación de eventos

Desde PowerShell, en la carpeta que contiene docker-compose.yml:

```
docker compose exec -T mosquitto mosquitto_sub -h localhost -t "turnos/#" -v
```

Ejecutar en una sola línea. Se detiene con Ctrl+C. Para probar, usar dos terminales: una suscrita y otra publicando. También puede utilizarse MQTT Explorer en localhost:1883. Una prueba de alta o cancelación debe contrastarse con la API o MySQL.

Fuentes del contrato: GeneradorReservasMQTT, SubscriberReservaMQTT, EventosTurnosMQTT, ProcesadorSolicitudes, FacturadorAutomatico, ReservaDTO, CancelarTurnoDTO, DisponibilidadDTO, AgendaDTO, SolicitudReservaMensaje, CancelarReservaMensaje y MqttPublicadorServicio del proyecto proporcionado.